SLIDE 1: Introdução à Arquitetura de Software

Introdução

- O princípio básico da arquitetura de software é que cada sistema é construído para satisfazer metas de negócio de uma organização;
- Ponte de metas abstratas e o sistema concreto;

O que é Arquitetura de Software?

- **É o conjunto de estruturas** necessárias para raciocinar sobre o sistema. Estas estruturas incluem elementos de software, relacionamentos entre eles, e as propriedades de ambos. (Não é somente documentação ou diagramas estáticos)

Estruturas arquiteturais podem ser organizadas em três categorias principais:

- **Estruturas de Componentes & Conectores; Estruturas de Módulo; Estruturas de Alocação.**

Arquitetura é uma abstração. Descreve o sistema em alto nível. Elementos principais. Visão simplificada. Ponto de referência. Compreensão do sistema.

Arquitetura versus Design (Se complementam)

- A arquitetura foca em decisões estruturais de alto impacto. Design trata dos detalhes de implementação.

Estruturas Arquiteturais: Um sistema pode ser descrito por múltiplas estruturas (ex.: módulos, componentes, processos).

Estrutura de Componentes e Conectores

- **Componentes:** unidades principais de execução ou armazenamento. Exemplos: processos, serviços, bancos de dados, UI.
- **Conectores:** mecanismos de interação entre componentes. Exemplos: chamadas de procedimento, mensagens, eventos, protocolos.
- **Essa visão mostra como os elementos interagem em tempo de execução Fundamental para analisar desempenho, confiabilidade e segurança.**
- **Exemplo - Estrutura de Serviço:**

Mostra como serviços independentes são organizados e interagem. Cada serviço encapsula funcionalidade específica do sistema. Comunicam-se via interfaces bem definidas (APIs, mensagens, eventos).

- **Exemplo - Estrutura de Concorrência**

Mostra como componentes ou processos executam em paralelo. Destaca threads, processos, pools de trabalho e mecanismos de sincronização.

Estrutura de Módulos

- Mostra o sistema como um conjunto de unidades de implementação. Cada módulo agrupa elementos de software que mudam juntos. Enfatiza a decomposição estática do sistema. Exemplos: pacotes, namespaces, bibliotecas, subsistemas. Importante para atribuir responsabilidades e organizar o trabalho da equipe.
- **Todos abaixo são (Subtipo da estrutura de módulos).**

Exemplo - Estrutura de Decomposição

- Mostra o sistema como uma árvore hierárquica de módulos. Foca na atribuição de responsabilidades e na organização estática. Entender em partes menores.

Exemplo - Estrutura de Uso

- Mostra as dependências de uso entre módulos.

Exemplo - Estrutura em Camadas

- Organiza o sistema em níveis hierárquicos (camadas). Cada camada oferece serviços à camada imediatamente superior.
- Exemplos comuns: Apresentação → Negócio → Persistência → Infraestrutura Facilita substituição de implementações e isolamento de responsabilidades.

Exemplo - Estrutura de Classes

- Mostra classes e suas relações de herança, dependência e composição. Representa a organização estática em sistemas orientados a objetos. Ajuda a analisar reuso, polimorfismo e extensibilidade.

Estrutura de Modelo de Dados

- Foca nos dados persistentes e suas relações. Mostrar entidades, atributos e associações (ex.: ER diagramas).

Estrutura de Alocação

- Mostra como os elementos de software são mapeados para o ambiente. Relaciona software a hardware, sistemas de arquivos, equipes ou rede. Exemplos: Mapeamento de processos em servidores.
- Distribuição de módulos em times de desenvolvimento. Organização em diretórios ou arquivos. Fundamental para entender implantação, desempenho e responsabilidades organizacionais

Exemplo - Estrutura de Implantação

- Mostra como o software é mapeado para o hardware e ambientes físicos. Cada nó de implantação representa um servidor, dispositivo ou contêiner. Destaca interações de rede, dependências e restrições de recursos.

Exemplo - Estrutura de Implementação

- Mostra como o código-fonte e artefatos estão organizados. Exemplos: diretórios, pacotes, bibliotecas, módulos de build.

Exemplo - Estrutura de Atribuição de Trabalho

- Mostra como as tarefas e responsabilidades são distribuídas entre equipes.

Categoria	Foco	Principais Questões
Component-and-Connector (C&C)	Comportamento em tempo de execução	Como os elementos interagem? Como lidar com concorrência e falhas?
Estruturas de Módulos	Organização estática do software	Como o sistema é decomposto? Quais dependências existem? Como atribuir responsabilidades?
Estruturas de Alocação	Mapeamento para ambiente físico	Onde o software será implantado? Como recursos e equipes suportam o sistema?

Definindo a Arquitetura

- As arquiteturas mais simples reduzem a complexidade e o risco.

Quais Estruturas Escolher? Depende

- Performance e falhas → Component-and-Connector
- Organização estática e responsabilidades → Estruturas de Módulos
- Implantação, hardware e equipes → Estruturas de Alocação.

A Importância da Arquitetura de Software

- Permite direcionar os atributos de qualidade do sistema.
- As decisões feitas na arquitetura permitem gerenciar as mudanças à medida que o sistema evolui.
- Define as restrições para implementações subsequentes. Dita a estrutura da organização. Base para um desenvolvimento incremental. Combinar componentes.

Viabilizando Atributos de Qualidade

- A arquitetura influencia diretamente os atributos de qualidade do sistema.
- Decisões arquiteturais podem habilitar ou limitar atributos como: Desempenho, Segurança, Confiabilidade e Manutenibilidade.
- **Escolha de padrões arquiteturais influência:** Escalabilidade de uma aplicação web Segurança em sistemas financeiros Facilidade de manutenção e evolução.
- **Sistemas sofrem mudanças frequentes:** A arquitetura fornece base para raciocinar.

Arquitetura Predizendo os Atributos de Qualidade do Sistema

Métodos para Predição de Qualidade

- **Análise de arquitetura:** avaliar estruturas e padrões para estimar impacto.
- **Simulações e protótipos:** testar para validar desempenho e confiabilidade.
- **Medições históricas e métricas:** usar dados de sistemas similares para previsão.

Decisões de Design Precoce: Arquitetura captura decisões críticas desde o início. Muitas decisões arquiteturais são difíceis ou caras de alterar depois da implementação.

Benefícios de Decisões Arquiteturais Precoce: Previsibilidade do sistema. Prototipagem e validação antecipada. Comunicação.

Restrições na Implementação: Arquitetura define limites e regras para a construção do sistema.

Influência na Estrutura Organizacional..

Facilitando o Desenvolvimento Incremental.

Estimativas de Custo e Cronograma.

Modelo Transferível e Reutilizável.

Incorporação de Elementos Desenvolvidos Independentemente.

Arquitetura guia escolhas de design consistentes.

Base para Treinamento.

Leis da Arquitetura de Software

- Tudo em uma Arquitetura de Software é um trade-off (Escolher uma opção em detrimento de outra).
- "Por quê?" é mais importante do que o "Como?" - Documentação
-

SLIDE 2: Compreendendo os Atributos de Qualidade

Atributos de Qualidade: Características do sistema que impactam sua qualidade além das funcionalidades. Determinam como o sistema se comporta.

Importância: Direcionam decisões arquiteturais. Determinam a capacidade do sistema de evoluir. Orientam testes e validação do sistema.

- **Funcionalidade responde** “o que o sistema faz” **Qualidade responde** “como o sistema se comporta”

Classificação dos Atributos de Qualidade

Externos (visíveis aos usuários): desempenho, usabilidade, segurança, confiabilidade

Internos (visíveis aos desenvolvedores): modificabilidade, testabilidade, portabilidade

Exemplos de Atributos de Qualidade

- Desempenho: tempo de resposta, throughput
- Disponibilidade: tempo de atividade, recuperação de falhas
- Segurança: confidencialidade, integridade, autenticação
- Modificabilidade: facilidade de adaptação a mudanças

Trade-offs entre Atributos Nem todos os atributos podem ser maximizados ao mesmo tempo.

Cenários de Qualidade

- É crucial descrever atributos de forma clara e mensurável, evitando termos vagos como “rápido” ou “seguro”.

Estrutura de um Cenário de Qualidade:
1. **Fonte de estímulo** (quem/que causa o evento)
2. **Estímulo** (o evento em si)
3. **Artefato** (parte do sistema afetada)
4. **Ambiente** (condições sob as quais ocorre)
5. **Resposta** (o que o sistema faz)
6. **Medição da resposta** (critério de sucesso)

Categorias:
1. **Uso**: descrevem o comportamento durante a execução.
2. **Mudança**: lidam com modificações no sistema.
3. **Falha**: tratam respostas a falhas.

Táticas de Qualidade
Quality Attribute Workshop (QAW): Técnica para capturar e priorizar atributos de qualidade junto aos stakeholders.
Attribute-Driven Design (ADD): Método de projeto arquitetural orientado por atributos de qualidade.
Táticas de Qualidade: Soluções de baixo nível para alcançar atributos de qualidade.
Exemplos de Táticas para **Desempenho:** Redução da demanda: compressão, cache.
Gerenciamento de concorrência: filas, agendamento. **Segurança**: Prevenção: autenticação forte, autorização. **Detecção**: monitoramento de logs, IDS.
Padrões e Métodos de Avaliação

- Padrões arquiteturais conhecidos já incorporam táticas de qualidade. Exemplos: Camadas (Layers): apoia modificabilidade. Cliente-Servidor: separa responsabilidades. Microservices: promove escalabilidade e independência.

Avaliando Arquiteturas : Validar se a arquitetura atende aos cenários críticos.
Métodos formais:

- **SAAM**: Mais simples e rápido. Foco principal em modificabilidade.
- **ATAM**: Mais abrangente e detalhado. Considera múltiplos atributos de qualidade. Foco na identificação de trade-offs entre eles.

Métodos informais: prototipagem, simulação, revisões.

Qualidade de Software: Qualidade é uma propriedade emergente: Resulta da interação entre vários elementos do sistema.

SLIDE 3: Implantação e Eficiência

O que é Implantação ? É a capacidade de colocar um sistema em uso em seu ambiente operacional de forma eficiente e confiável.

- Envolve não apenas a instalação inicial, mas também atualizações, configurações e rollbacks. **Objetivo**: Entregar valor ao cliente de forma rápida e segura, minimizando o tempo de inatividade e o risco.

Táticas de Implantação

- **Automação de Implantação**: Usar scripts e pipelines para reduzir erros manuais. (Objetivo: Tornar o processo de implantação repetível, confiável e rápido através de pipelines de CI/C).
- **Estratégias de Liberação:** Adotar padrões para reduzir o risco do lançamento.
- **Gerenciamento de Configuraçã**o: Usar Feature Toggles e Infraestrutura como Código.
- **Monitoramento Contínuo**: Observar o sistema durante e após a implantação.

- **Estratégia: Blue-Green Deployment:** Mantém dois ambientes de produção idênticos ("Blue" e "Green"). O tráfego ao vivo vai para um (Blue), a nova versão é implantada no outro (Green). Após a validação, o tráfego é roteado para o Green.
- **Estratégia: Canary Release:** A nova versão é liberada para um pequeno subconjunto de usuários. Se o monitoramento indicar estabilidade, a liberação é expandida gradualmente.
- **Tática: Feature Toggles:** Permite ativar ou desativar funcionalidades em tempo de execução sem a necessidade de uma nova implantação.

Rollback: Reverter para a versão anterior estável quando uma falha é detectada. É a estratégia mais segura para recuperação imediata.
Rollforward: Corrigir o problema com uma nova versão (um patch). Exige um processo de implantação muito rápido e confiança na correção.

Métricas Chave: DORA Metrics (Frequência, Lead Time, Taxa de Falha, MTTR).

Eficiência (Performance)

Eficiência (ou Desempenho) é a capacidade do sistema de fornecer respostas em tempo hábil, usando uma quantidade apropriada de recursos, sob diferentes cargas de trabalho. **Por que é importante?** Afeta diretamente a experiência do usuário e os custos operacionais.

Métricas Chave: Tempo de Resposta (Latência). Vazão (alta Throughput é ruim): Quantas requisições o sistema processa por segundo. Utilização de recursos.
Táticas:
- **Controle de Recursos:** Gerenciar como os recursos são consumidos.
- **Redução de Demanda:** Diminuir a quantidade de trabalho a ser feito.
- **Gerenciamento de Concorrência:** Processar múltiplas requisições simultaneamente para melhorar o throughput.
- **Balanceamento de Carga**: Distribuir a carga de trabalho entre múltiplos servidores para evitar gargalos.
- **Gerenciamento de Cache:** Reduzir a demanda sobre recursos caros armazenando resultados de operações frequentes em uma camada de acesso rápido.

Identificando Gargalos: Profiling
- Profiling é a análise do programa para determinar quais partes consomem mais tempo ou recursos.

Eficiência e Escalabilidade
- **Escalabilidade Vertical:** Aumentar os recursos de um único servidor (mais CPU/RAM). Simples, mas com limites.
- **Escalabilidade Horizontal:** Adicionar mais servidores. Requer uma arquitetura que suporte distribuição (ex: serviços stateless).
-

SLIDE 4: Integrabilidade e Modificabilidade

Integrabilidade - Definição: A capacidade de fazer componentes de software, desenvolvidos separadamente, cooperarem. Envolve a integração de um ou mais componentes em um sistema existente. Lida com os custos e riscos técnicos de tarefas de integração, previstas e imprevistas.
- **A dificuldade da integração depende de - Tamanho:** O número de dependências entre os componentes. - **Distância:** A complexidade de resolver as diferenças em cada dependência. Essas dependências podem ser:

- **Dimensão 1: Distância <u>Sintática</u>** - Os elementos que cooperam devem concordar sobre o número e o tipo dos elementos de dados compartilhados. **Exemplo:** Um componente envia um inteiro, mas o outro espera um ponto flutuante.
- **Dimensão 2: Distância <u>Semântica</u> de Dados** - Os elementos devem concordar sobre a semântica dos dados. Mesmo que o tipo de dado seja o mesmo, a interpretação pode ser diferente.
- **Dimensão 3: Distância Semântica Comportamental -** Os elementos devem concordar sobre o comportamento, especialmente em relação aos estados e modos do sistema (inicialização, desligamento, recuperação).
- **Dimensão 4: Distância <u>Temporal</u> -** Os elementos devem concordar sobre suposições de tempo.
- **Dimensão 5: Distância de Recursos -** Os elementos devem concordar sobre as suposições de recursos compartilhados. **Exemplos**: Um componente exige acesso exclusivo a um dispositivo.

Táticas: focam em limitar dependências, adaptar e coordenar.

1. Encapsular

- Objetivo → **Limitar dependências**
- Ação → **Criar uma interface explícita (tudo tem que passar por ela)**
- Benefício → **Evita o efeito dominó**

2. Usar um Intermediário

- Objetivo → **Limitar dependências**
- Ação → **Quebrar a ligação direta entre componentes**
- Exemplo → **Repositório compartilhado**

3. Aderir a Padrões

- Objetivo → **Limitar dependências**
- Ação → **Adotar padrões de mercado (IEEE, ISO) ou regras locais**
- Benefício → **Reduz problemas de comunicação**

4. Adaptar Interface

- Objetivo → **Adaptar**
- Ação → **Adicionar ou ocultar funções sem alterar a API original ou o código base**

5. Orquestrar

- Objetivo → **Coordenar**
- Ação → **Usar um (orquestrador) para chamar os serviços, mantendo eles soltos (desacoplados)**

Padrões de Arquitetura

1. Arquitetura Orientada a Serviços (SOA)

- O que é? → Componentes distribuídos (fornecem ou consomem serviços)
- Benefícios → Serviços genéricos, reutilizáveis e independentes (acessados via rede)

2. Padrão Wrapper

- O que faz? → Encapsula um componente criando uma nova abstração (transforma dados ou controle)
- Funções principais:
    - → Traduzir uma interface
    - → Ocultar uma interface
    - → Preservar a interface base

Modificabilidade

- **Definição:** A capacidade de um sistema permitir que alterações sejam feitas de forma eficiente, com baixo custo e risco. A maior parte do custo de um sistema de software ocorre após o seu lançamento inicial.
- **"Sabores" da Modificabilidade - Escalabilidade**: Acomodar mais recursos (vertical/horizontal). **Variabilidade:** Suportar a produção de variantes de um sistema (linha de produto). **Portabilidade**: Mudar o software para rodar em outra plataforma. **Independência de Localização**: Mudar a localização de componentes em tempo de execução.

Medidas de Complexidade: Coesão e Acoplamento

- **Coesão:** Mede a "unidade de propósito" de um módulo. Alta coesão é desejável.
- **Acoplamento:** Mede a dependência entre módulos. Baixo acoplamento é desejável.
- Reduzir o acoplamento e aumentar a coesão são os principais objetivos das táticas de modificabilidade.

Táticas: visam aumentar a coesão, reduzir o acoplamento e adiar o vínculo.

1. Dividir Módulo

- Objetivo → **Aumentar a coesão**
- Ação → **Pegar um módulo "faz-tudo" e quebrar em módulos menores e super focados.**

2. Encapsular

- Objetivo → **Reduzir o acoplamento**
- Ação → **Esconderclara.**
- Exemplo → **Você pode mudar toda a lógica interna de um componente sem afetar quem está usando ele de fora.**

3. Usar um Intermediário

- Objetivo → **Reduzir o acoplamento**
- Ação → **Colocar um "meio-campo" para que os módulos não se conheçam diretamente.**
- Exemplo → **Em vez do módulo A chamar o C direto, o A chama o B (intermediário), e o B aciona o C.**

4. Restringir Dependências

- Objetivo →**Reduzir o acoplamento**
- Ação → **Criar regras estritas de quem pode falar com quem.**
- Exemplo Clássico → **Arquitetura em Camadas (uma camada só pode conversar com a camada imediatamente abaixo dela).**

5. Adiar Vínculo

- Objetivo → **Ganhar flexibilidade adiando decisões.**
- Ação → **Não deixar valores "chumbados" (hardcoded) no código. Deixe o sistema/computador definir isso na hora que rodar.**

Táticas de Modificabilidade

1. Vínculo em Tempo de Configuração

- Ação → **Ler valores de um arquivo de configuração na hora que o sistema inicia.**
- Benefício → **Alterar o sistema sem precisar mexer ou recompilar o código.**
- Exemplo → **Mudar a URL do banco de dados ou credenciais de acesso.**

2. Polimorfismo

- Ação → **Vincular diferentes implementações de uma interface enquanto o sistema roda (tempo de execução).**

Padrões

1. Cliente-Servidor

- Estrutura → **Um servidor central atende a vários clientes.**
- Modificabilidade → **Baixo acoplamento (evoluem separados).**
- Exemplo Prático → **Um frontend web construído em React.js (cliente) consumindo uma API REST de um backend feito em Java/Spring Boot (servidor).**

2. Plug-in

- Estrutura → **Um núcleo principal (microkernel) + componentes extras (plug-ins) conectados por interfaces fixas.**
- Modificabilidade → **Equipes diferentes podem criar plug-ins de forma totalmente independente do núcleo.**

3. Camadas (Layers)

- Estrutura → **Divisão em níveis. A comunicação é de cima para baixo (unidirecional).** Modificabilidade → **Você pode alterar o banco de dados (camada inferior) sem quebrar a interface de usuário (camada superior).**

4. Publicador-Assinante

- Estrutura → **Publicadores mandam mensagens para "Tópicos" e os Assinantes escutam esses tópicos.** Modificabilidade → **Totalmente solto (desacoplado). Você adiciona ou remove assinantes dinamicamente.**

SLIDE 5: Segurança e Segurança Funcional

Segurança Funcional: - Definição: A capacidade de um sistema de evitar entrar em estados que causem ou levem a danos, ferimentos ou perda de vidas. **Foco:** Prevenir falhas que tenham consequências catastróficas no mundo físico.

Táticas de Segurança Funcional:

1. EVITAR (Prevenir a falha)

- Objetivo → Não deixar o sistema entrar em perigo.
- Substituição → Trocar proteção de software por hardware.
- Modelo Preditivo → Prever o problema com antecedência

2. DETECTAR

- Objetivo → Perceber que o sistema entrou em estado perigoso.
- Timeout → Estourar o tempo limite para detectar falhas de omissão e timing.
- Monitoramento de Condição.
- Verificação de Sanidade → Validar se a operação ou os dados fazem sentido lógico.
- Comparar as saídas de elementos replicados para ver se batem.

3. CONTENÇÃO

- Objetivo → Limitar os danos quando o sistema já entrou em um estado inseguro. Divide-se em 3 subcategorias:

A. Redundância

- Ação → Usar componentes redundantes para continuar operando.
- Replicação → Clones idênticos.

B. Limitar Consequências

- Ação → Reduzir a funcionalidade de forma planejada e segura.
- Abortar → Interromper a operação insegura na força bruta antes que cause danos.

C. Barreira

- Ação → Conter problemas para que não se propaguem pelo sistema.
- Firewall → Limitar o acesso a recursos específicos (processadores, memória, rede).
- Interlock → Controlar o sequenciamento correto de eventos para proteger os componentes.

Categoria: Recuperação

- Objetivo → Colocar o sistema de volta em um estado seguro após uma falha.
- Rollback → Reverter o sistema para uma cópia anterior de um estado que sabidamente funcionava.
- Reparar Estado → Corrigir o erro internamente e continuar a execução do sistema.
- Reconfiguração → Remapear a arquitetura lógica para usar apenas os recursos físicos que ainda estão funcionando.

Padrões de Segurança

1. Sensores Redundantes

- Estrutura → Replicar qualquer sensor que seja crítico para a segurança.
- Benefício → Protege o sistema caso um sensor único falhe.

2. Monitor-Atuador

- Estrutura → Um componente "Monitor" checa se o comando do "Controlador" faz sentido antes de enviá-lo para a máquina/hardware.
- Benefício → Separa quem faz o cálculo de quem faz a verificação.

3. Segurança Separada

- Estrutura → Dividir o sistema claramente entre o que é crítico e o que não é crítico para a segurança.
- Benefício → Fica mais barato certificar o software, pois apenas a parte crítica passará pela validação rigorosa.

Segurança (Security)

Segurança é a medida da capacidade do sistema de proteger dados e informações contra acesso não autorizado, ao mesmo tempo em que fornece acesso a pessoas e sistemas autorizados.

Tríade CID

- **Confidencialidade:** A propriedade de que dados ou serviços são protegidos contra acesso não autorizado.
- **Integridade**: A propriedade de que dados ou serviços não estão sujeitos a manipulação não autorizada.
- **Disponibilidade**: A propriedade de que o sistema estará disponível para uso legítimo

Privacidade: se refere a limitação de acesso a informações pessoais identificáveis (PII).

Táticas de Segurança: O objetivo principal é controlar a resposta a um ataque.

1. Detectar Ataques Objetivo → Perceber que a tentativa de ataque está acontecendo.

2. Resistir a Ataques Objetivo → Dificultar a vida do invasor e blindar o sistema.

3. Reagir a Ataques Objetivo → Tomar atitudes imediatas com o ataque já em andamento.

4. Recuperar de Ataques Objetivo → Voltar à operação segura e aprender com a invasão.

Padrões de Segurança

1. Intercepting Validator (Validador Interceptador)

- Ação → Insere um elemento "wrapper" (envoltório) no meio do caminho, entre a origem e o destino das mensagens.
- Responsabilidade Principal → Verificar a integridade da mensagem (se ela não foi alterada).

2. Sistema de Prevenção de Intrusão (IPS)

- Ação → Funciona como um elemento autônomo vigiando a rede. Ele identifica e analisa atividades suspeitas (se for aceitável, deixa passar; se for ameaça, bloqueia e relata).

SLIDE 6: Testabilidade e Usabilidade

Testabilidade

- Refere-se à facilidade com que o software pode ser levado a demonstrar suas falhas por meio de testes.
- **Modelo de Teste:** Um programa processa uma entrada e produz uma saída. Um oráculo (humano ou computacional) decide se a saída está correta. O estado interno do programa também pode ser observado pelo oráculo para verificar sua correção.

Táticas de Testabilidade: O objetivo das táticas de testabilidade é promover testes mais fáceis, eficientes e capazes.
As categorias são:

- **Controlar e Observar o Estado**
- **Ideia geral:** facilitar ver e controlar o que está acontecendo dentro do sistema durante testes.
- Interfaces Especializadas.Gravação/Reprodução. Estado em um só lugar.
- Asserções: verificações no código que avisam quando algo está errado.
- **Limitar a Complexidade**
- **Ideia geral:** deixar o sistema mais simples e previsível.
- Complexidade Estrutural.

Usabilidade

A usabilidade se preocupa com a facilidade com que um usuário pode realizar uma tarefa desejada e o tipo de suporte que o sistema oferece.

- **Áreas da Usabilidade**
- **Aprendizado:** sistema fácil para iniciantes
- **Eficiência:** ajuda o usuário a fazer tarefas mais rápido
- **Erros:** reduzir impacto de erros do usuário
- **Adaptação:** sistema se ajusta ao usuário (ou vice-versa)
- **Satisfação:** gerar confiança e boa experiência

Iniciativa (quem "toma ação")

- **Usuário:** usuário inicia (ex: cancelar ação)
- **Sistema:** sistema age sozinho (ex: mostrar progresso)
- **Mista:** os dois (mais comum)

Cenário de Usabilidade

- **Fonte:** usuário
- **Estímulo:** necessidade (fazer tarefa)
- **Artefato:** interface (GUI, CLI)
- **Ambiente:** uso real ou configuração
- **Resposta:** sistema ajuda + dá feedback
- **Medida:** tempo, erros, aprendizado, satisfação

Táticas de Usabilidade Objetivo: dar **ajuda + feedback ao usuário**

- **Apoiar o Usuário**
- **Cancelar:** parar ação
- **Undo (Desfazer):** voltar estado
- **Pausar/Retomar:** continuar depois
- **Agregar:** aplicar ação em grupo
- **Apoiar o Sistema**
- **Modelo da Tarefa:** prever o que o usuário quer (ex: autocomplete)
- **Modelo do Usuário:** adaptar ao nível do usuário
- **Modelo do Sistema:** mostrar estado (ex: barra de progresso)
- **PADRÕES DE USABILIDADE**
- **Padrão MVC (Model-View-Controller)** Ideia: separar o sistema em 3 partes:
- **Model**: lógica de negócio (dados e regras)
- **View**: interface (o que o usuário vê)
- **Controller:** recebe ações do usuário, processa e comunica Model e View.

Padrão Observer: Ideia: quando algo muda, outros são avisados automaticamente. **Padrão Mement**o: Ideia: salvar estados para poder voltar depois (undo).